हेलो! आपका स्वागत है **Unit 14: Events** की complete study class में.

मैंने आपके workspace में, unit 14 events folder के अंदर ये complete study files write कर दी हैं:

- Study Guide File: chapter14_events.md
- Code Examples Sandbox: events_examples.html
- Practice Sandbox File: events_practice.html

चलिए अब Events, event object properties, bubbling, capturing aur event delegation concepts ko simple Hinglish explanation ke sath samajhna shuru karte hain.

PART 1: EVENT BASICS & LISTENERS

Events webpage elements par user ki activities aur interactions (clicks, keyboard strokes, form submissions) ke triggers hote hain.

1. Event Kya Hai?

What is it? Events webpage par hone wali user activity indicators dynamic signals hain.

- **click:** Element click tracker.
- **input:** Keystrokes characters inputs tracker.
- **change:** State inputs update changes on blur checks.
- **submit:** Form submission events.
- **keydown/keyup:** Keyboard press logs.

Why is it needed? Static document layouts ko user actions par react karwane ke liye (e.g. menu expand, modal open).

How does it work? We register an event listener using addEventListener() linking callback actions: element.addEventListener("type", callback)

Simple example

javascript

```
let btn = document.getElementById("myBtn");

btn.addEventListener("click", () => {
   console.log("Button click success!");
});
```

---

2. Event Object (e)

What is it? Browser event trigger hone par callback parameter input variable pass karta hai jise Event Object e bolte hain. Isme event trigger elements, key value records information properties store rehti hain.

Important properties:

- **e.target:** Actual element targeted (jahan action originate hua).
- **e.type:** Click, keydown events type string.
- **e.key:** Keys typed indicator strings parameters.

Simple example

javascript

```
document.addEventListener("keydown", (e) => {
   console.log("Pressed key label: " + e.key);
});
```

Output Pressed key label: a (when 'a' key is pressed)

PART 2: EVENT PROPAGATION FLOW

Events nodes hierarchies coordinates checks loops bubble.

1. Event Bubbling vs Capturing

What is it? Event execution sequence hierarchies flow models:

- **Event Bubbling:** target element (child) se start hokar parent and outer grandparent levels tak execute sequence path up (Default).
- **Event Capturing:** outer parent coordinates se trickle down target child element coordinate parameters.

Example

javascript

```
// Bubbling (default)
child.addEventListener("click", () => console.log("child"));
// Capturing (pass true as 3rd param)
parent.addEventListener("click", () => console.log("parent"), true);
```

---

2. preventDefault() vs stopPropagation()

What is it?

- **preventDefault():** Stops default browser tasks validations (reloads on form submit, link page transitions redirects).
- **stopPropagation():** Halts bubbling chain sequences upwards.

Simple example preventDefault

javascript

```
let form = document.querySelector("form");

form.addEventListener("submit", (e) => {
   e.preventDefault(); // page reload stops
});
```

Simple example stopPropagation

javascript

```
let btn = document.querySelector("button");

btn.addEventListener("click", (e) => {
   e.stopPropagation(); // stops parent click triggering
});
```

---

3. Event Delegation

What is it? Har single child tags par listener separate registers targets checks ignore karke, unke shared common parent element par single listener bind targets mappings setup compile check triggers properties.

Why is it useful? Performance optimizations, dynamic additions child elements automatic configurations matching checks.

Simple example

html

```
<ul id="menu">
   <li>Home</li>
```

```
    <li>Profile</li>
</ul>
```

```
javascript
let menu = document.getElementById("menu");
menu.addEventListener("click", (e) => {
    if (e.target.tagName === "LI") {
        console.log("Selected option: " + e.target.textContent);
    }
});
```

🧠 QUICK REVISION SUMMARY

- **Event listeners:** registered via addEventListener.
- **Event object:** holds active elements targets properties pointers (target, key).
- **Bubbling:** moves child to parent. Capturing moves parent to child.
- **preventDefault:** cancels browser default behaviors.
- **stopPropagation:** halts bubbling events upwards parent nodes.
- **Event delegation:** Single parent listener handles events for all child nodes using event target references.

📝 IMPORTANT DEFINITIONS (INTERVIEW-READY)

1. **Event Bubbling:** Default event path model traversing from child target element to parent nodes.
2. **Event Delegation:** Pattern handling multiple child listeners using a single parent container listener.
3. **event.target:** Reference to the actual element that triggered the event.
4. **event.currentTarget:** Reference to the element to which the active event listener is attached.
5. **Event Capturing (Trickling):** Event propagation path traversing downwards from parent elements to the target child.

💬 **IMPORTANT INTERVIEW QUESTIONS & ANSWERS**

Q1. Difference between event.target and event.currentTarget?

Ans: e.target is the actual element that triggered the event (the origin). e.currentTarget is the element to which the active event listener is attached.

Q2. Explain Bubbling vs Capturing.

Ans: Bubbling triggers events from the child target element upward to the parent (default). Capturing triggers events from parent container downward to the child target element (enabled by passing true as the third argument in addEventListener).

Q3. Difference between preventDefault() and stopPropagation()?

Ans: preventDefault() stops browser default behavior (like link redirects). stopPropagation() stops event bubbling upward through the parent element hierarchy.

Q4. Predict output:

javascript

```
document.addEventListener("click", (e) => {
   console.log(e.type);
});
```

Ans: "click".

Q5. What is Event Delegation and why use it?

Ans: Attaching a single event listener to a parent element to handle events for all children (existing and future dynamic ones), saving memory and processing resources.

Q6. Predict output:

html

```
<form>
  <button type="submit">Go</button>
</form>
```

javascript

```
document.querySelector("form").addEventListener("submit", (e) => {
   // without preventDefault
```

});

Ans: Form submits and browser page reloads instantly, resetting the application state.

Q7. How do you detect which key was pressed during keydown events?

Ans: By reading the event.key property inside the event callback function.

Q8. What does e.stopPropagation() return?

Ans: It returns undefined.

Q9. Can you register multiple event listeners on the same element?

Ans: Yes. E.g., adding both "click" and "mouseover" listeners to a single button.

Q10. Predict output:

javascript

```
let btn = document.querySelector("button");
btn.addEventListener("click", () => console.log("A"));
btn.addEventListener("click", () => console.log("B"));
```

Ans: "A" "B" (Both callbacks run sequentially in order of registration).

Q11. Predict output of input event type:

javascript

```
input.addEventListener("input", () => console.log("typed"));
```

Ans: Prints "typed" on every single keystroke.

Q12. What does change event type do?

Ans: Triggers only when input loses focus (blur) after its value changes, or when select dropdown items are selected.

Q13. Predict output:

javascript

```
document.body.addEventListener("click", (e) => {
   console.log(e.target);
});
// user clicks background body directly
```

Ans: Logs <body> element node representation.

Q14. How to remove event listeners in JS?

Ans: By calling element.removeEventListener("type", functionName). Note: Callback function must have a named reference.

Q15. Does stopPropagation prevent other listeners on the same element?

Ans: No. It only prevents bubbling to parents. To stop other listeners on the same element, call e.stopImmediatePropagation().

Q16. Predict output:

javascript

```
let tag = document.querySelector("input");

tag.addEventListener("change", (e) => {
   console.log(e.target.value);
});
```

Ans: Logs the updated input string values as soon as user types and focuses away (blur).

Q17. What is default third parameter value in addEventListener?

Ans: false (meaning Bubbling is active by default).

Q18. Predict output:

javascript

```
document.addEventListener("click", (e) => {
   e.preventDefault();
});

// user clicks checkboxes
```

Ans: Checkboxes do not toggle state on clicking (default checked toggle is disabled).

Q19. What is Event Bubbling origin path top target?

Ans: The window object.

Q20. Predict output:

javascript

```
let button = document.createElement("button");

button.click(); // programmatic click
```

Ans: Triggers any registered click event handlers programmatically.

💻 PRACTICE QUESTIONS

Question 1: Click Event Logger

- **Question:** Alert button text values on click event.
- **HTML:** <button class="alertBtn">Button A</button>
- **JavaScript:**

javascript

```
const btn = document.querySelector(".alertBtn");
btn.addEventListener("click", (e) => {
  console.log(e.target.textContent); // logs "Button A"
});
```

- **Explanation:** Reads content from e.target.textContent on click event trigger.

Question 2: Stop bubbling parent alert

- **Question:** Child click should not trigger parent click event logging.
- **JavaScript:**

javascript

```
const parent = document.getElementById("parent");
const child = document.getElementById("child");
parent.addEventListener("click", () => console.log("Parent block"));
child.addEventListener("click", (e) => {
  e.stopPropagation(); // stops bubbling
  console.log("Child block");
});
```

- **Explanation:** Calls e.stopPropagation() inside child click to prevent event bubbling to parent listener.

Question 3: Dynamic elements delegation check

- **Question:** Create event delegation on a list to delete elements dynamically on click.
- **JavaScript:**

javascript

```
const list = document.getElementById("parentList");

list.addEventListener("click", (e) => {
   if (e.target.tagName === "LI") {
      e.target.remove(); // removes clicked list item
   }
});
```

- **Explanation:** Places a single listener on parent container, filters clicked tags by checking tagName === "LI", and removes target elements dynamically.